

IOAI Skill Patterns: Logic and Methods That Make Tasks Easier

A compact Overleaf-ready guide for practice and contests

Core idea. In IOAI, a strong solution is often not “train a bigger model”. The useful skill is to recognize the hidden structure: the metric, the data geometry, the matching constraint, the judge model, or the allowed compute budget. The 2025 official repository lists tasks such as Radar, Chicken Counting, Concepts, Restroom Icon Matching, Antique Painting Authentication, and Pixel Efficiency [1]. The official syllabus also explicitly includes programming, classical ML, clustering, PCA, deep learning, U-Net segmentation, transfer learning, and augmentation [2]. Use the patterns below as a checklist before coding.

Skill pattern	When to use it	First move
Metric-to-loss	Score is weighted, asymmetric, or based on a final aggregate	Write the scoring formula; make your training loss imitate it.
Density-map thinking	Need a count, not exact boxes	Predict nonnegative density; sum over pixels; optimize count error.
Visualize and cluster	Few labels but obvious structure in features	Plot PCA/t-SNE; try KNN, DBSCAN, or a simple threshold.
Feature matching	Need to pair images/items, not classify each independently	Build a similarity matrix from cheap descriptors; enforce one-to-one.
Hungarian method	A global assignment is required	Convert similarity to cost; call <code>linear_sum_assignment</code> .
Judge-model probing	The evaluator contains a model you can inspect or imitate	Query/call it sparingly; search inputs that make it confident.
Postprocess and sanity-check	Outputs violate obvious constraints	Clamp, normalize, threshold, calibrate, or sort labels consistently.

1. Make the metric your loss

Use when: the score values different errors differently. **Do not train on a default loss until you know what the leaderboard is measuring.**

Radar example. In IOAI 2025 Radar, each test case gave six radar-feed images and asked for a segmentation map. The unofficial writeup notes that non-background pixels were worth far more than background pixels; a strong improvement was to use U-Net with a cross-entropy class weight that made background much less important [3]. The transferable trick is:

$$L = - \sum_{p \in \text{pixels}} w_{y_p} \log P_{\theta}(y_p | x)_p, \quad \text{choose } w_c \propto \text{score value of class } c.$$

Checklist. If the metric says “missing a rare class hurts a lot”, then increase its loss weight. If the metric only cares about a sum, add a loss term on the sum. If the metric clips or thresholds, validate with the same clipping/thresholding.

Common failure. A high pixel accuracy can be terrible if the rare class is the score driver. Always compute a local validation score that matches the contest metric, not just a standard training loss.

2. For counting, predict a density map

Use when: the task asks “how many?” but labels are spatial or object locations are crowded. Chicken Counting provided density maps whose pixel sum equals the true chicken count; test outputs were also density maps, and scoring compared the predicted sum with the true count [4]. The official reference solution used a feature extractor plus U-Net-style decoder and trained against density maps; its notebook also applied a final ReLU to keep predictions nonnegative [5].

Practical loss trick. Combine a local map loss with a global count loss:

$$L = \lambda_1 \|\hat{D} - D\|_1 + \lambda_2 \left| \sum_{i,j} \hat{D}_{ij} - \sum_{i,j} D_{ij} \right|, \quad \hat{D} \leftarrow \max(0, \hat{D}).$$

The first term teaches localization; the second matches the score. ReLU/clamping prevents impossible negative animals. A validation printout like `true_sum`, `pred_sum`, `abs_error` catches mistakes much faster than looking only at average loss.

3. Visualize first, then cluster or threshold

Use when: labels are scarce, but samples may form geometric groups. In Antique Painting Authentication, samples were 5-D points with only a few labels, and real/fake points were known to be separated; high-scoring approaches visualized with PCA/t-SNE and then used KNN or DBSCAN, with DBSCAN on t-SNE reaching about 99 points in the unofficial writeup [6].

Contest routine.

1. Plot the first two coordinates, PCA, and t-SNE/UMAP if available.
2. Color the known labels; ask: are clusters, moons, or lines visible?
3. Try the simplest classifier that matches the picture: threshold, KNN, DBSCAN, or label propagation.
4. Stabilize label orientation. If an unsupervised method returns clusters 0/1, map cluster IDs to labels using validation/known points, not by hope.

Why it works. In olympiad-style AI tasks, the hidden trick is often that the official dataset has a deliberately discoverable structure. Visualization is a low-cost way to find that structure before spending time on neural models.

4. Use feature matching before deep learning

Use when: the task is “find the matching item” rather than “predict a label”. For Restroom Icon Matching, the goal was to match a cropped icon to the uncropped image from the same restroom and opposite gender. An unofficial 100-point approach used color histograms to reduce candidates to likely same-restroom pairs, then used a classifier to decide gender [7].

Recipe. Build a matrix S_{ij} where row i is a query and column j is a candidate. Start with cheap features: color histogram, edge histogram, CLIP embedding, ResNet embedding, template matching score, or cosine similarity. Only after this should you train a heavier model to break ties.

Fast win. If images from the same source share lighting, wall color, border style, or compression artifacts, classical descriptors can be more reliable than a small neural model trained from scratch.

5. Hungarian method for global one-to-one matching

Use when: each item must be used once, or local best choices collide. The linear sum assignment problem is minimum-weight bipartite matching; SciPy’s `linear_sum_assignment` returns an assignment minimizing the total cost and also handles rectangular matrices [9].

If S_{ij} is a similarity score between query i and candidate j , convert it to a cost $C_{ij} = -S_{ij}$ for maximization:

$$\min_X \sum_{i,j} C_{ij} X_{ij}, \quad X_{ij} \in \{0,1\}, \quad \sum_j X_{ij} \leq 1, \quad \sum_i X_{ij} \leq 1.$$

```
from scipy.optimize import linear_sum_assignment
# S[i, j] = similarity between query i and candidate j
row, col = linear_sum_assignment(-S) # maximize similarity
pairs = list(zip(row, col))
```

Use it after constructing S from histograms/embeddings. This turns many fragile greedy pairings into one globally consistent answer.

6. Search and augment under the compute budget

Use when: the answer is an input transformation: crop, mask, prompt, threshold, or augmentation. In Pixel Efficiency, the task was to find a small crop, no more than 6.25% of the image, that a provided CLIP model would classify correctly;

the same CLIP model was available, but the notebook had to process 700 images in under eight minutes, so it could not call CLIP too many times per image [8]. The reference idea in the writeup processed many candidate masks in parallel by using attention-mask style batching [8].

Programming patterns.

- Replace nested brute force with coarse-to-fine search: try a grid, keep top candidates, refine around them.
- Cache features for repeated crops or text prompts.
- Batch candidate transformations, especially on GPU.
- Use integral images for fast rectangle sums when the score is additive.
- Add safe augmentations: flips, small crops, brightness/contrast, test-time averaging. Keep a validation scoreboard so augmentation never becomes random guessing.

7. Postprocess with constraints

Use when: the raw model output violates facts you know. Examples: density cannot be negative; class probabilities should respect a mask; labels from clustering may be flipped; assignment outputs should be unique; segmentation maps may need small-object cleanup.

Three-pass contest workflow.

1. **Understand the metric.** Write a 20-line local scorer before model training.
2. **Exploit structure.** Visualize, cluster, match, search, or add a constraint. Try the cheapest method that could work.
3. **Make it robust.** Validate on splits, seed everything, test edge cases, and postprocess outputs into the required format.

Final mnemonic: metric $\rightarrow$ structure $\rightarrow$ constraint $\rightarrow$ code speed. Most IOAI improvements come from one of these four, not from blindly enlarging the model.

References

- [1] IOAI-official. *IOAI 2025 tasks and datasets repository*. <https://github.com/IOAI-official/IOAI-2025>
- [2] IOAI. *IOAI 2025 Syllabus*. <https://ioai-official.org/wp-content/uploads/2025/06/Syllabus-2025-Final.pdf>
- [3] IOAI Writeup. *IOAI 2025 Day 1 - Radar*. <https://ioai-writeup.github.io/posts/d1p1-radar/>
- [4] IOAI Writeup. *IOAI 2025 Day 1 - Chicken Counting*. <https://ioai-writeup.github.io/posts/d1p2-chicken/>
- [5] IOAI-official. *Chicken Counting Reference Solution notebook*. https://github.com/IOAI-official/IOAI-2025/blob/main/Individual-Contest/Chicken_Counting/Chicken_Counting_Solution.ipynb
- [6] IOAI Writeup. *IOAI 2025 Day 2 - Antique*. <https://ioai-writeup.github.io/posts/d2p2-antique/>
- [7] IOAI Writeup. *IOAI 2025 Day 2 - Restroom*. <https://ioai-writeup.github.io/posts/d2p1-restroom/>
- [8] IOAI Writeup. *IOAI 2025 Day 2 - Pixel*. <https://ioai-writeup.github.io/posts/d2p3-pixel/>
- [9] SciPy. *linear_sum_assignment documentation*. https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linear_sum_assignment.html